TECHNICAL PROJECT REPORT

Zephyr RTOS Code-Completion Fine-Tuning

Data engineering, LoRA adaptation, held-out evaluation, C++ remediation, and five-GPU scaling

Prepared for	Release	Status	Date
Technical assessment review	v1.0.0	Complete and published	17 August 2026

Executive conclusion

The pipeline is reproducible and the final 7B adapter demonstrates positive held-out lift on both broad Zephyr code and the dedicated C++ benchmark. The original 1.5B adapter remains useful because it delivered the largest general-domain gain, but it regressed on C++ and is therefore published with an explicit limitation.

Deliverable	Result
Data pipeline	13,697 files inventoried; 12,823 retained
Evaluation	512 general + 62 dedicated C++ examples
Best balanced model	Qwen2.5-Coder-7B LoRA, checkpoint 100
Publishing	GitHub v1.0.0 + two Hugging Face adapter repositories

1. Objective and model design

The assessment objective was to adapt a compact code model to idiomatic Zephyr RTOS completion and demonstrate measurable improvement over the unchanged base model. Zephyr is a real-time operating system for constrained embedded devices; its code combines kernel primitives, device drivers, devicetree configuration, Kconfig guards, architecture ports, tests, and hardware-specific macros. A credible model must therefore learn both ordinary C/C++ syntax and Zephyr-specific conventions.

Training used left-context completion pairs. The prompt contains source code before a cut point; the target contains the reference continuation. Prompt labels are masked with -100, so the loss applies only to completion tokens and EOS. Low-Rank Adaptation (LoRA) freezes the base model and learns compact rank-16 updates (alpha 32, dropout 0.05) for attention and feed-forward projections. This reduced trainable parameters to 18.46 million for the 1.5B model and 40.37 million for the 7B model.

2. Data engineering and leakage control

Stage	Files / examples	Key control
Inventory	13,697 source files	Hashes, size, language, SPDX, duplicates
Cleaning	12,823 retained	Generated, vendored, duplicate, oversized, hex-heavy and empty exclusions
Split	80% / 10% / 10%	3,501 parent-directory groups kept intact
Completion data	64,935 / 4,774 / 4,517	Train / validation / evaluation examples
Benchmarks	512 general; 62 C++	Evaluation remained untouched during tuning

Zephyr v4.4.1 was pinned to commit 1f6485eca25431b5ff27ce9a754218c9e559bbbb. Exact hashes removed duplicate content, and parent-directory grouping prevented neighboring files from crossing splits. Validation confirmed unique IDs and fingerprints, the 1,024-token limit, directory-group isolation, source-hash isolation, and 500 independent token-count spot checks with zero mismatches.

A tokenizer audit found that concatenating prompt and completion text before byte-pair encoding changed the first boundary token in about 17% of sampled records. At inference time, this merge cannot happen because the prompt is already tokenized. The corrected pipeline tokenizes both parts separately, concatenates token IDs, preserves the complete target, masks the prompt, and checks decode round trips.

Explicit C++ was extremely scarce: only 69 cleaned .cpp/.hpp files and 183 raw training completions. Priority-aware repetition increased the effective C++ share from 4.57% to 19.92% without modifying validation or evaluation data.

3. Training and multi-GPU execution

The 1.5B adapter was trained on one TITAN RTX and verified through smoke tests, completion-only masking checks, validation, PEFT integrity checks, and full held-out evaluation. Version-sensitive trainer options were detected at runtime so unsupported optional fields could be omitted without weakening required settings.

For 7B, device_map='auto' solved memory capacity by sharding layers across five GPUs, but it did not provide parallel example throughput: one batch still traversed the layer sequence. The faster design used Accelerate with DistributedDataParallel (DDP): five processes, one full model replica per TITAN RTX, different data shards, and synchronized LoRA gradients. Batch 2 per GPU reached 4.661 examples/s in smoke testing versus 1.302 examples/s for the auto-sharded configuration. The published 7B artifact is checkpoint 100, taken forward after validation-only screening and then evaluated on the untouched held-out benchmarks.

4. Held-out results

Model / benchmark	NLL reduction	Accuracy delta	First-line delta	Edit-similarity delta
1.5B / general	+19.17%	+3.10 pt	+5.27 pt	+5.11 pt
1.5B / C++	-15.51%	-0.43 pt	+1.61 pt	-0.55 pt
7B / general	+7.12%	+0.33 pt	+2.74 pt	+2.08 pt
7B / C++	+7.08%	+0.46 pt	+4.84 pt	+1.19 pt

The 1.5B adapter produced the strongest broad Zephyr adaptation: general NLL fell from 0.950232 to 0.768027 and token accuracy rose from 78.88% to 81.98%. However, its C++ NLL increased from 0.917017 to 1.059281. Reporting this regression was important: an aggregate improvement would have hidden failure on the assessment's most specialized subset.

The 7B checkpoint was the better balanced result. General NLL fell from 0.740071 to 0.687411, while C++ NLL fell from 0.653255 to 0.606977. C++ token accuracy increased from 84.55% to 85.01%, first-line exact match from 59.68% to 64.52%, and edit similarity from 36.50% to 37.69%. General exact match also increased from 0.59% to 3.12%.

Teacher-forced NLL and token accuracy measure probability assigned to the reference continuation; greedy exact, first-line, and edit metrics measure generated output. Using both prevents a likelihood-only improvement from being overstated as a practical completion gain. All base/adapter comparisons used the same pinned model revision, tokenizer, prompts, targets, and decoding settings.

5. Reproducibility and published artifacts

- Pinned model revisions: Qwen2.5-Coder-1.5B df3ce67c... and Qwen2.5-Coder-7B 0396a761...
- Deterministic seed 20260717, manifest chain, leakage assertions, smoke mode, JSON run summaries, persistent logs, and adapter-integrity checks.
- Source, configurations, scripts, results summary, and this report: GitHub repository
- Versioned project snapshot: GitHub release v1.0.0
- Published adapters: 1.5B LoRA and 7B LoRA checkpoint 100